Laborator Python: funcții, metode și importuri pe web

Laborator pentru studenții din anul I

Nume și prenume: Iurco Mihai

Grupa: SI-261 **Data:** 08.10.2026

Prezentarea laboratorului

În acest laborator veți scrie programe Python care comunică cu un site web real și veți exersa trei concepte de bază: **importurile**, **metodele** și **funcțiile**. La final veți avea propriul modul Python, care generează un raport despre un site web.

După acest laborator veți putea:

- Să importați module din biblioteca standard și module externe și să explicați diferența dintre ele.
- Să apelați metode și să citiți atribute ale obiectelor create de alt cod (răspunsuri, șiruri de caractere, dicționare).
- Să scrieți propriile funcții, cu parametri, valori implicite, valori returnate, docstring-uri și tratarea erorilor.
- Să împărțiți codul într-un modul propriu și să îl importați dintr-un alt fișier.

Ce trebuie să știți înainte: variabile, instrucțiuni `if`, bucle `for`, liste și dicționare.

Site-ul țintă: toate exercițiile de citire folosesc `https://cybercor.org`. Exercițiile care *trimit* date folosesc `https://httpbin.org`, un serviciu public creat pentru testarea cererilor.

Termeni web esențiali

Termen	Ce înseamnă	Exemplu
URL	Adresa unei resurse pe web	`https://cybercor.org/robots.txt`
Cerere (request)	Mesajul pe care programul vostru îl trimite unui server	`GET /`
Răspuns (response)	Răspunsul serverului la cerere	cod de stare + antete + corp
Cod de stare (status code)	Un număr care arată cum s-a încheiat cererea	`200` OK, `301` mutat, `404` negăsit
Antet (header)	O pereche nume–valoare cu informații suplimentare	`Content-Type: text/html`
Corp (body)	Conținutul propriu-zis returnat	codul HTML al paginii
HTML	Limbajul de marcare în care sunt scrise paginile web	`<title>CYBERCOR</title>`

Importuri, metode și funcții, pe scurt

- Un **import** aduce în programul vostru cod scris de altcineva: `import requests`.

- O **metodă** este o funcție care aparține unui obiect și se apelează cu punct: `response.json()`, `text.lower()`.
- O **funcție** este un bloc de cod cu nume, reutilizabil, pe care îl scrieți voi: `def get_title(html): ...`.

Configurare și reguli

Înainte de exercițiul 1, instalați o bibliotecă externă și creați un director pentru laborator.

1. Verificați versiunea Python cu `python --version` (3.9 sau mai nouă).
2. Instalați biblioteca `requests`: `pip install requests`.
3. Creați un director numit `web_lab` și lucrați în el. Salvați fiecare parte într-un fișier separat: `part1_imports.py`, `part2_methods.py` și așa mai departe.
4. Începeți fiecare fișier cu acest șablon:

```
# Laborator: funcții, metode și importuri pe web
# Student: <numele vostru>

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
```

Reguli de lucru

Lucrați cu un site web real, de care depind oameni reali. Respectați aceste reguli la fiecare exercițiu:

- **Folosiți mereu un timeout.** Fiecare cerere primește `timeout=TIMEOUT`, ca programul să nu se blocheze la nesfârșit.
- **Faceți pauze între cereri.** Când trimiteți mai multe cereri una după alta, puneți `time.sleep(1)` între ele.
- **Doar citire.** Către cybercor.org trimiteți doar cereri `GET` și `HEAD`. Tot ce trimite date (`POST`, parametri de interogare, antete personalizate pe care vreți să le verificați) merge către httpbin.org.
- **Nicio buclă fără limită.** Nu scrieți niciodată o buclă `while True:` care trimite cereri.
- **Rămâneți pe țintă.** Accesați doar URL-urile indicate în exerciții. Nu scanați alte site-uri.

Încălcarea acestor reguli poate duce la blocarea adresei IP a întregii săli, de aceea respectarea lor face parte din notă.

Partea 1: Importuri (exercițiile 1–8)

Scop: învățați diferitele moduri de a aduce cod în program. Fișier: `part1_imports.py`.

Exercițiul 1. **Primul import.** Scrieți `import requests` și afișați `requests.__version__`. Apoi scrieți `import urllib.request`. Într-un comentariu, explicați de ce `requests` a avut nevoie de `pip install`, iar `urllib` nu.

```
import requests
print(requests.__version__)
import urllib.request
```

Exercițiul 2. Două stiluri de import. Descărcați pagina `BASE_URL` cu `requests.get()` și afișați codul de stare. Apoi rescrieți același script folosind `from requests import get`. Într-un comentariu, scrieți câte un avantaj al fiecărui stil.

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)

from requests import get
response = get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)
```

Exercițiul 3. Alias-uri. Importați requests cu un alias: `import requests as rq`. Descărcați pagina principală cu `rq.get()`. Într-un comentariu, spuneți când un alias face codul mai ușor de citit și când îl face mai greu de citit.

```
import requests as rq
response = rq.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)
```

Exercițiul 4. Doar biblioteca standard. Descărcați pagina principală folosind doar `urllib.request.urlopen()`. Afișați atributul `.status` al răspunsului și primele 200 de caractere din corp.

> **Indiciu:** `.read()` returnează octeți (bytes). Folosiți `.decode("utf-8")` ca să îi transformați într-un șir de caractere.

```
import urllib.request
import urllib.error

req = urllib.request.Request(
    BASE_URL,
    headers={"User-Agent": "WebLab-Popescu Ion"},
)

try:
```

```
    with urllib.request.urlopen(req, timeout=TIMEOUT) as resp:
        print(resp.status)
        body = resp.read().decode("utf-8")
        print(body[:200])
except urllib.error.HTTPError as e:
    print("Serverul a răspuns cu eroare:", e.code, e.reason)
```

Exercițiul 5. **Priviți în interiorul unui modul.** Rulați `print(dir(requests))`. Alegeți trei nume din rezultat și scrieți, într-un comentariu, dacă fiecare este o funcție, o clasă sau un modul.

```
print(dir(requests))
```

Exercițiul 6. **Citiți documentația.** Rulați `help(requests.get)`. Găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

```
help(requests.get)
response = requests.get(BASE_URL, timeout=TIMEOUT)
```

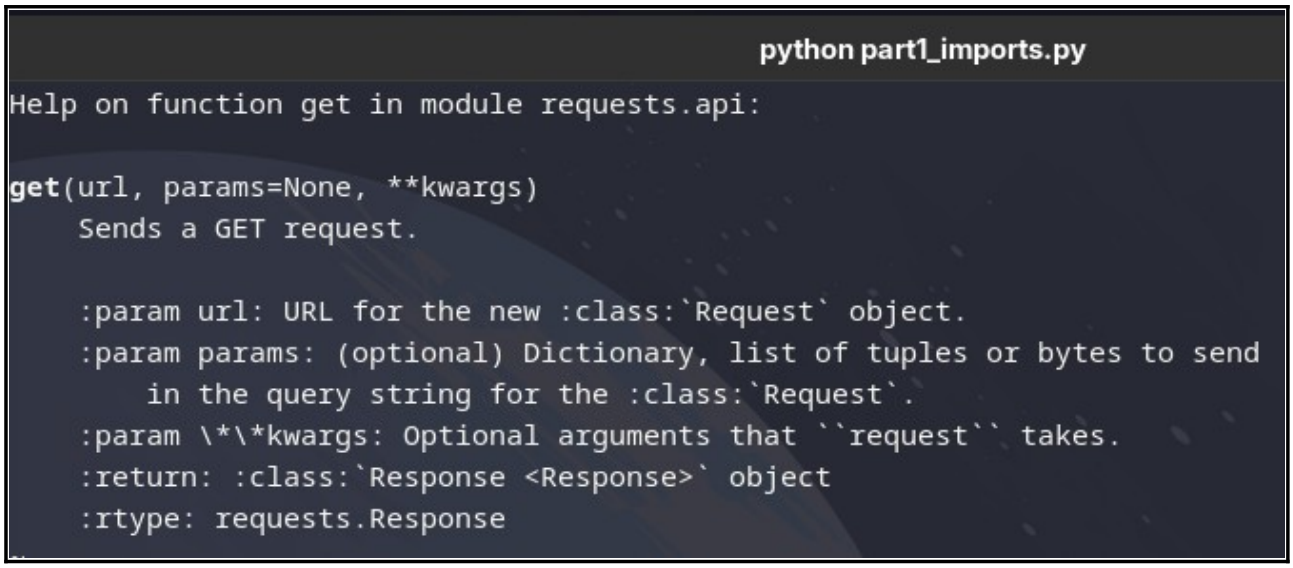
Exercițiul 7. **Cronometrarea unei cereri.** Importați `time` și măsurați cât durează o cerere către pagina principală, apelând `time.perf_counter()` înainte și după. Comparați rezultatul cu `response.elapsed`.

```
import time
start = time.perf_counter()
response = requests.get(BASE_URL, timeout=TIMEOUT)
end = time.perf_counter()
print(f"perf_counter: {end - start:.3f} s")
print(f"response.elapsed: {response.elapsed.total_seconds():.3f} s")
```

Exercițiul 8. **Când un import eșuează.** Scrieți un script care încearcă `import bs4` într-un bloc `try` / `except ImportError`. Dacă modulul lipsește, afișați: `Instalați modulul cu: pip install beautifulsoup4`.

```
try:
    import bs4
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```





> **Verificați-vă:** care este diferența dintre un modul, un pachet și o bibliotecă?
>
> *Un modul este un singur fișier .py care conține cod reutilizabil (funcții, clase, variabile). Exemple: time, webtools.py. Un pachet este un director care conține mai multe module organizate împreună, de obicei*

> *cu un fișier **init**.py. Exemple: requests, urllib (urllib.request și urllib.parse sunt module din pachetul urllib). O bibliotecă este un termen general pentru o colecție de cod gata făcut, pe care o folosim în programele noastre. Ea poate conține un singur modul sau mai multe pachete. Exemple: biblioteca standard Python, biblioteca requests.*

Partea 2: Metode și atribute ale obiectelor web (exercițiile 9–20)

Scop: învățați să folosiți instrumentele pe care obiectele le au deja. Un **atribut** este o valoare (`response.status_code`); o **metodă** este o acțiune pe care o apelați cu paranteze (`response.json()`). Fișier: `part2_methods.py`.

Exercițiul 9. **Elementele de bază ale răspunsului.** Descărcați pagina principală și afișați `response.status_code`, `response.ok`, `response.url` și `response.encoding`. Lângă fiecare linie, scrieți un comentariu care spune dacă este un atribut sau o metodă.

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)
print(response.ok)
print(response.url)
print(response.encoding)
```

Exercițiul 10. **Tratarea erorilor cu o metodă.** Apelați `response.raise_for_status()` pe pagina principală. Apoi trimiteți o cerere către `BASE_URL + "/this-page-does-not-exist"` și prindeți excepția `requests.HTTPError`. Afișați un mesaj prietenos în loc să lăsați programul să se oprească cu eroare.

```
response.raise_for_status()
time.sleep(1)
try:
    r = requests.get(BASE_URL + "/this-page-does-not-exist",
timeout=TIMEOUT)
    r.raise_for_status()
except requests.HTTPError as e:
    print("Pagina nu a fost găsită, dar programul continuă. Detalii:",
e)
```

Exercițiul 11. **Toate antetele.** Parcurgeți `response.headers.items()` și afișați fiecare antet în forma `Nume: valoare`.

```
for name, value in response.headers.items():
    print(f"{name}: {value}")
```

Exercițiul 12. Metode de dicționar. Folosiți `response.headers.get()` pentru a afișa antetele `Server` și `Content-Type`, cu `"lipsește"` ca valoare implicită. Apoi încercați `response.headers.get("content-type")`, cu litere mici. Ce observați?

```
print(response.headers.get("Server", "lipsește"))
print(response.headers.get("Content-Type", "lipsește"))
print(response.headers.get("content-type", "lipsește"))
```

Exercițiul 13. Înlănțuirea metodelor de șir. Numărați de câte ori apare cuvântul `cyber` pe pagina principală, cu `response.text.lower().count("cyber")`. Într-un comentariu, explicați de ce puteți înlănțui `.lower()` și `.count()`.

```
print(response.text.lower().count("cyber"))
```

Exercițiul 14. Găsirea titlului. Folosiți `.find()` și felierea (slicing) pe `response.text` pentru a extrage textul dintre `<title>` și `</title>`. Curățați-l cu `.strip()` și afișați-l.

> **Indiciu:** start = html.find("<title>") + len("<title>")

```
html = response.text
start = html.find("<title>") + len("<title>")
end = html.find("</title>")
print(html[start:end].strip())
```

Exercițiul 15. Numărarea liniilor. Împărțiți codul HTML în linii cu `.splitlines()`. Afișați câte linii sunt și lungimea celei mai lungi linii, folosind `max(lines, key=len)`.

```
lines = html.splitlines()
print(len(lines), "linii")
print(len(max(lines, key=len)), "caractere în cea mai lungă linie")
```

Exercițiul 16. Verificarea HTTPS. Folosiți `.startswith()` pentru a verifica dacă `response.url` începe cu `https://`. Afișați `Conexiune securizată` sau `Conexiune nesecurizată`.

```
if response.url.startswith("https://"):
    print("Conexiune securizată")
else:
    print("Conexiune nesecurizată")
```

Exercițiul 17. Urmărirea redirecționărilor. Trimiteți o cerere către `http://cybercor.org` (HTTP simplu, fără S). Parcurgeți `response.history` și afișați codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

```
time.sleep(1)
r = requests.get("http://cybercor.org", timeout=TIMEOUT)
for step in r.history:
    print(step.status_code, step.url)
print("URL final:", r.url)
```

Exercițiul 18. HEAD versus GET. Trimiteți o cerere către pagina principală cu `requests.head()` și una cu `requests.get()`. Comparați `len(response.content)` pentru ambele și explicați diferența într-un comentariu.

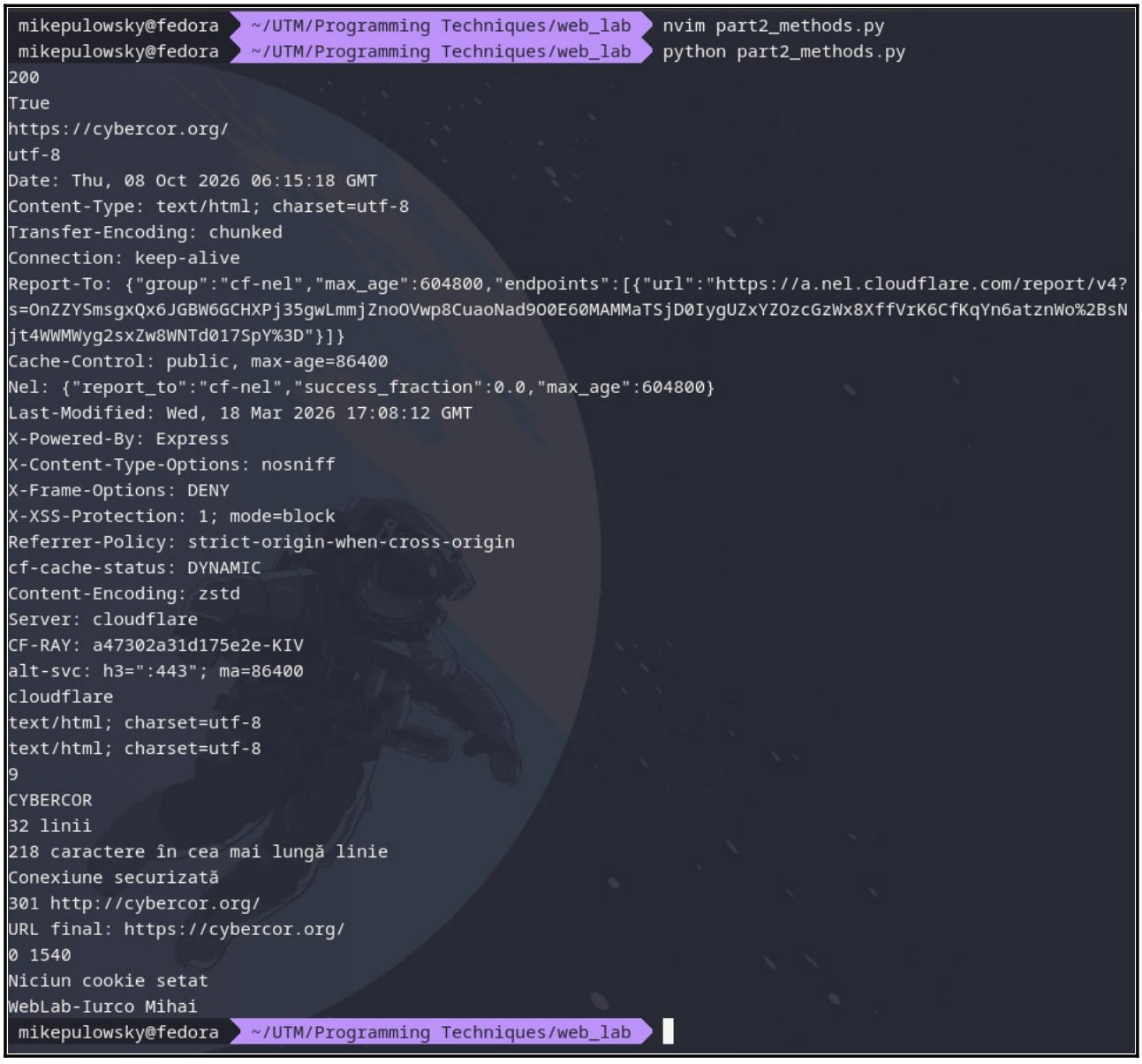
```
time.sleep(1)
h = requests.head(BASE_URL, timeout=TIMEOUT)
g = requests.get(BASE_URL, timeout=TIMEOUT)
print(len(h.content), len(g.content))
```

Exercițiul 19. Cookie-uri. Parcurgeți `response.cookies` și afișați, pentru fiecare cookie, atributele `.name` și `.secure`. Dacă nu există cookie-uri, afișați `Niciun cookie setat`.

```
if len(response.cookies) == 0:
    print("Niciun cookie setat")
else:
    for c in response.cookies:
        print(c.name, c.secure)
```

Exercițiul 20. Sesiuni. Creați un obiect `requests.Session()` și setați un `User-Agent` personalizat cu `session.headers.update({"User-Agent": "WebLab-<numele vostru>"})`. Trimiteți o cerere către `ECHO_URL + "/headers"` și confirmați în rezultat că antetul a fost trimis.

```
time.sleep(1)
session = requests.Session()
session.headers.update({"User-Agent": "WebLab-Iurco Mihai"})
r = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
print(r.json()["headers"]["User-Agent"])
```



> **Verificați-vă:** atât `response.text`, cât și `response.json()` vă dau corpul răspunsului. De ce unul are paranteze, iar celălalt nu?
>
> *response.text este un atribut (property): o valoare deja disponibilă, pe care doar o citim, de aceea nu are paranteze. response.json() este o metodă: o acțiune care trebuie executată, pentru că interpretează (parsează) corpul răspunsului ca JSON și îl transformă într-un dicționar sau o listă. Parantezele apelează metoda, ea poate primi argumente și poate ridica o eroare (JSONDecodeError) dacă corpul nu este JSON valid.*

Partea 3: Scrierea propriilor funcții (exercițiile 21–34)

Scop: organizați codul în funcții reutilizabile. De acum înainte, fiecare exercițiu este o funcție pe care o apelați cel puțin o dată. Fișier: `part3_functions.py`.

```
def get_status(url):
  """Returnează codul de stare HTTP pentru adresa url."""
  response = requests.get(url, timeout=TIMEOUT)
  return response.status_code
```

Exercițiul 21. Prima funcție. Scrieți `fetch(url)`, care returnează obiectul răspuns. Apelați-o pentru `BASE_URL` și afișați codul de stare.

```
def fetch(url):
    """Returnează obiectul răspuns pentru adresa url."""
    return requests.get(url, timeout=TIMEOUT)

print(fetch(BASE_URL).status_code)
```

Exercițiul 22. Returnarea unei singure valori. Scrieți `get_status(url)`, care returnează doar codul de stare. Apelați-o pentru `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

```
def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

for path in ["/", "/robots.txt", "/sitemap.xml"]:
    print(path, get_status(BASE_URL + path))
    time.sleep(1)
```

Exercițiul 23. Parametri impliciți. Modificați funcția în `fetch(url, timeout=10)`. Apelați-o o dată cu valoarea implicită și o dată cu `timeout=3`.

```
def fetch(url, timeout=10):
    """Returnează răspunsul pentru url, cu un timeout configurabil."""
    return requests.get(url, timeout=timeout)

print(fetch(BASE_URL).status_code)
time.sleep(1)
print(fetch(BASE_URL, timeout=3).status_code)
```

Exercițiul 24. O singură sarcină pe funcție. Scrieți `get_title(html)`, care primește un șir HTML și returnează titlul paginii, refolosind logica din exercițiul 14. Observați că această funcție nu face nicio operație de rețea: descărcarea și analiza sunt acum separate.

```
def get_title(html):
    """Returnează titlul paginii HTML sau un șir gol dacă lipsește."""
    start = html.find("<title>")
    end = html.find("</title>")
    if start == -1 or end == -1:
        return ""
    return html[start + len("<title>"):end].strip()

print(get_title(fetch(BASE_URL).text))
```

Exercițiul 25. Docstring-uri. Adăugați un docstring funcției `get_title` și afișați-l cu `help(get_title)`.

```
help(get_title)
```

Exercițiul 26. Adnotări de tip. Adăugați adnotări de tip (type hints) funcțiilor, de exemplu `def get_status(url: str) -> int:`. Apelați `get_status(123)` și observați ce se întâmplă. Vă opresc adnotările de tip?

```
def get_status(url: str) -> int:
```

```
    """Returnează codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

try:
    get_status(123)
except Exception as e:
    print("Eroare:", type(e).__name__, e)
```

Exercițiul 27. Fără opriri neașteptate. Scrieți `page_exists(url)`, care returnează `True` sau `False`. Folosiți `try` / `except requests.RequestException`, astfel încât funcția să returneze `False` în loc să se oprească cu eroare, chiar și pentru un URL precum `https://this-domain-does-not-exist.invalid`.

```
def page_exists(url: str) -> bool:
    """Returnează True dacă pagina răspunde cu succes, altfel False."""
    try:
        return fetch(url).ok
    except requests.RequestException:
        return False

print(page_exists(BASE_URL))
print(page_exists("https://this-domain-does-not-exist.invalid"))
```

Exercițiul 28. Returnarea unui dicționar. Scrieți `check_paths(base, paths)`, care primește o listă precum `["/", "/robots.txt", "/sitemap.xml"]` și returnează un dicționar `{path: status_code}`.

```
def check_paths(base: str, paths: list) -> dict:
    """Returnează un dicționar {cale: cod_de_stare} pentru fiecare
cale."""
    results = {}
    for path in paths:
        results[path] = get_status(base + path)
        time.sleep(1)
    return results
```

```
print(check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))
```

Exercițiul 29. Argumente cu nume. Scrieți `get_header(url, name, default="lipsește")`, care returnează valoarea unui singur antet. Apelați-o cu argumente cu nume: `get_header(BASE_URL, name="Server")`.

```
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Returnează valoarea unui antet sau valoarea implicită dacă lipsește."""
    return fetch(url).headers.get(name, default)


print(get_header(BASE_URL, name="Server"))
```

Exercițiul 30. Antete de securitate. Scrieți `security_headers(url)`, care verifică prezența acestor cinci antete și returnează `{antet: True/False}`:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Referrer-Policy`

```
SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]


def security_headers(url: str) -> dict:
    """Returnează {antet: True/False} pentru cele 5 antete de securitate."""
    headers = fetch(url).headers
    return {name: name in headers for name in SECURITY_HEADERS}


print(security_headers(BASE_URL))
```

Exercițiul 31. Funcții care apelează funcții. Scrieți `score_headers(results)`, care primește dicționarul din exercițiul 30 și returnează un șir precum `"3/5"`. Apelați-le împreună: `score_headers(security_headers(BASE_URL))`.

```
def score_headers(results: dict) -> str:
    """Returnează scorul antetelor de securitate sub forma
'prezente/total'."""
    return f"{sum(results.values())}/{len(results)}"


print(score_headers(security_headers(BASE_URL)))
```

Exercițiul 32. Două funcții, o sarcină. Scrieți `fetch_robots(base)`, care returnează textul fișierului `/robots.txt` sau `None` dacă fișierul lipsește. Apoi scrieți `disallowed_paths(robots_text)`, care returnează o listă cu toate valorile `Disallow:`. Asigurați-vă că funcționează și atunci când primește `None`.

```
def fetch_robots(base: str):
    """Returnează textul /robots.txt sau None dacă fișierul
lipsește."""
    try:
        r = fetch(base + "/robots.txt")
    except requests.RequestException:
        return None
    return r.text if r.status_code == 200 else None


def disallowed_paths(robots_text) -> list:
    """Returnează lista valorilor Disallow din robots.txt (listă goală
pentru None)."""
    if robots_text is None:
        return []
    paths = []
    for line in robots_text.splitlines():
        if line.lower().startswith("disallow:"):
            value = line.split(":", 1)[1].strip()
            if value:
```

```
                paths.append(value)
    return paths

print(disallowed_paths(fetch_robots(BASE_URL)))
print(disallowed_paths(None))
```

Exercițiul 33. Oricâte argumente. Scrieți `response_times(*urls)`, care returnează un dicționar `{url: secunde}`. Apelați-o cu două URL-uri.

```
def response_times(*urls) -> dict:
    """Returnează {url: secunde} pentru fiecare url primit."""
    times = {}
    for url in urls:
        start = time.perf_counter()
        fetch(url)
        times[url] = round(time.perf_counter() - start, 3)
        time.sleep(1)
    return times

print(response_times(BASE_URL, BASE_URL + "/robots.txt"))
```

Exercițiul 34. Argumente cu nume colectate. Scrieți `log(message, **details)`, care afișează mesajul urmat de fiecare detaliu. Exemplu: `log("verificat", url=BASE_URL, status=200)` afișează `verificat | url=https://cybercor.org | status=200`.

```
def log(message, **details):
    """Afișează mesajul urmat de fiecare detaliu, separate prin ' |
'."""
    parts = [message] + [f"{k}={v}" for k, v in details.items()]
    print(" | ".join(parts))

log("verificat", url=BASE_URL, status=200)
```

```
 mikepulowsky@fedora  ~/UTM/Programming Techniques/web_lab  nvim part3_functions.py
 mikepulowsky@fedora  ~/UTM/Programming Techniques/web_lab  python part3_functions.py
200
/ 200
/robots.txt 200
/sitemap.xml 200
200
200
CYBERCOR

Eroare: MissingSchema Invalid URL '123': No scheme supplied. Perhaps you meant https://123?
True
False
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
cloudflare
{'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content
-Type-Options': True, 'Referrer-Policy': True}
3/5
[]
[]
{'https://cybercor.org': 0.376, 'https://cybercor.org/robots.txt': 0.665}
verificat | url=https://cybercor.org | status=200
 mikepulowsky@fedora  ~/UTM/Programming Techniques/web_lab
```

```
Help on function get_title in module __main__:

get_title(html)
    Returnează titlul paginii HTML sau un șir gol dacă lipsește.
 Help on get_title line 1/4 (END) (press h for help or q to quit)
```

Verificați-vă: care este diferența dintre a afișa o valoare cu `print()` în interiorul unei funcții și a o returna cu `return`?

print() doar afișează valoarea pe ecran, pentru om. După afișare, valoarea se pierde, iar funcția returnează None, deci codul care a apelat funcția nu o poate folosi. return trimite valoarea înapoi către cel care a apelat funcția, care o poate salva într-o variabilă, o poate folosi în calcule sau o poate transmite altei funcții (de exemplu score_headers(security_headers(url))). În plus, return încheie execuția funcției. Regula generală: funcțiile calculează și returnează, iar afișarea se face în afara lor.

Partea 4: Alte module din biblioteca standard (exercițiile 35–43)

Scop: combinați mai multe importuri pentru a rezolva probleme mai mari. Puneți fiecare soluție într-o funcție. Fișier: `part4_modules.py`.

Modul	La ce îl folosiți aici
`urllib.parse`	Descompunerea și compunerea URL-urilor
`re`	Căutarea de tipare în text
`html.parser`	Citirea corectă a codului HTML

Modul	La ce îl folosiți aici
hashlib	Calcularea amprentei unei pagini
json	Salvarea datelor într-un fișier
socket	Aflarea adreselor IP și deschiderea conexiunilor
ssl + datetime	Citirea datei de expirare a unui certificat

Exercițiul 35. Descompunerea unui URL. Folosiți urllib.parse.urlparse() pe https://cybercor.org/path?x=1#top. Afișați .scheme, .netloc, .path, .query și .fragment.

```
p = urlparse("https://cybercor.org/path?x=1#top")
print(p.scheme, p.netloc, p.path, p.query, p.fragment)
```

Exercițiul 36. Compunerea URL-urilor. Folosiți urllib.parse.urljoin() pentru a transforma legăturile relative /about, contact.html și ../index.html în URL-uri complete, pornind de la BASE_URL.

```
for link in ["/about", "contact.html", "../index.html"]:
    print(urljoin(BASE_URL, link))
```

Exercițiul 37. Extragerea legăturilor. Folosiți re.findall(r'href="([^"]+)"', html) pentru a obține toate legăturile de pe pagina principală. Puneți codul într-o funcție extract_links(html), care returnează legăturile fără duplicate.

```
def extract_links(html: str) -> list:
    """Returnează legăturile (href) din html, fără duplicate, în
ordinea apariției."""
    links = re.findall(r'href="([^"]+)"', html)
    return list(dict.fromkeys(links))


html = requests.get(BASE_URL, timeout=TIMEOUT).text
print(extract_links(html))
```

Exercițiul 38. Interne sau externe? Scrieți split_links(links, domain), care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri. Folosiți mai întâi urljoin(), apoi urlparse().netloc.

```
def split_links(links: list, domain: str):
    """Împarte legăturile în (interne, externe) față de domeniul
dat."""
    internal, external = [], []
    for link in links:
        full = urljoin("https://" + domain, link)
        if urlparse(full).netloc == domain:
            internal.append(full)
        else:
            external.append(full)
    return internal, external


print(split_links(extract_links(html), "cybercor.org"))
```

Exercițiul 39. Propriile metode. Creați o clasă care moștenește `html.parser.HTMLParser` și suprascrieți metoda `handle_starttag()`, pentru a colecta atributul `src` al fiecărui tag `<img>`.

```
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # codul vostru aici
        pass
```

> **Indiciu:** `attrs` este o listă de perechi `(nume, valoare)`, de exemplu `[("src", "/logo.png"), ("alt", "Logo")]`. Expresia `dict(attrs).get("src")` vă dă valoarea atributului `src` sau `None`.

Cum testați exercițiul 39. Testați mai întâi parserul pe un fragment HTML scris de voi, pentru care știți dinainte rezultatul corect. Abia după aceea rulați-l pe site-ul real. Adăugați acest test sub clasa voastră:

```
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                    # imagine obișnuită
    "poza.jpg",                     # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Dacă testul trece, parserul vostru: găsește tag-urile `<img>` indiferent de majuscule, ignoră tag-urile `<img>` fără `src`, ignoră celelalte tag-uri (de exemplu `<a>`) și păstrează ordinea din pagină. Dacă `assert` eșuează, comparați lista afișată de `print()` cu lista așteptată.

Apoi rulați parserul pe pagina reală:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Verificați rezultatul în două moduri: comparați numărul găsit cu `response.text.lower().count("<img")` și deschideți pagina în browser, apoi apăsați Ctrl+U ca să vedeți codul sursă. O listă goală nu înseamnă neapărat o greșeală: unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`. Tocmai de aceea testul cu HTML cunoscut vine primul.

```
class ImageFinder(HTMLParser):
    """Colectează atributul src al fiecărui tag <img>."""

    def __init__(self):
        super().__init__()
        self.images = []
```

```
    def handle_starttag(self, tag, attrs):
        if tag == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)

test_html = """
<html><body>
<img src="/logo.png" alt="Logo">
<IMG SRC="poza.jpg">
<img alt="imagine fără src">
<img src="https://cdn.example.com/banner.webp" />
<a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)
assert finder.images == [
    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")

response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
print("<img în text:", response.text.lower().count("<img"))
```

Exercițiul 40. **Amprenta paginii.** Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

```
 def page_fingerprint(url: str) -> str:
    """Returnează amprenta SHA-256 a conținutului paginii."""
    return hashlib.sha256(requests.get(url,
timeout=TIMEOUT).content).hexdigest()


a = page_fingerprint(BASE_URL)
time.sleep(1)
b = page_fingerprint(BASE_URL)
print(a)
print(b)
print(a == b)
```

Exercițiul 41. **Salvarea în JSON.** Salvați antetele paginii principale în `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Apoi încărcați fișierul înapoi cu `json.load()` și afișați un antet din el.

```
with open("headers.json", "w", encoding="utf-8") as f:
    json.dump(dict(response.headers), f, indent=2)
with open("headers.json", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded.get("Content-Type"))
```

Exercițiul 42. **Interogare DNS.** Scrieți `resolve(hostname)`, care returnează `socket.gethostbyname(hostname)`. Afișați adresa IP a domeniului `cybercor.org`.

```
def resolve(hostname: str) -> str:
    """Returnează adresa IPv4 a unui nume de domeniu."""
    return socket.gethostbyname(hostname)
```

```
print(resolve("cybercor.org"))
```

Exercițiul 43. Expirarea certificatului. Scrieți `cert_days_left(hostname)`, care se conectează la portul 443, citește certificatul și returnează câte zile mai sunt până la expirarea lui.

> **Indiciu:** folosiți `ssl.create_default_context()`, `socket.create_connection((hostname, 443))`, `context.wrap_socket(sock, server_hostname=hostname)` și `getpeercert()["notAfter"]`. Convertiți data cu `ssl.cert_time_to_seconds()`.

```
def cert_days_left(hostname: str) -> int:
    """Returnează numărul de zile rămase până la expirarea
certificatului TLS."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as
sock:
        with context.wrap_socket(sock, server_hostname=hostname) as
tls:
            cert = tls.getpeercert()
    expires = ssl.cert_time_to_seconds(cert["notAfter"])
    return int((expires - time.time()) // 86400)


print(cert_days_left("cybercor.org"))
```

```
mikepulowsky@fedora ~/UTM/Programming Techniques/web_lab nvim part4_modules.py
mikepulowsky@fedora ~/UTM/Programming Techniques/web_lab python part4_modules.py
https cybercor.org /path x=1 top
https://cybercor.org/about
https://cybercor.org/contact.html
https://cybercor.org/index.html
['/assets/index-DlIqjleG.css']
(['https://cybercor.org/assets/index-DlIqjleG.css'], [])
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
0 imagini găsite
<img în text: 0
c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
True
text/html; charset=utf-8
104.21.91.71
55
mikepulowsky@fedora ~/UTM/Programming Techniques/web_lab
```



> **Verificați-vă:** în exercițiul 39, cine apelează handle_starttag()? Voi nu o apelați niciodată direct.
>
> *Metoda handle_starttag() este apelată chiar de parser (clasa HTMLParser), din interiorul metodei feed(). Când feed() parcurge codul HTML și întâlnește un tag de deschidere, apelează automat handle_starttag(tag, attrs) pentru fiecare tag găsit. Noi doar definim (suprascriem) metoda, iar HTMLParser o apelează atunci când are nevoie de ea. Acest principiu se numește callback: nu o apelăm noi, ci parserul ne apelează pe noi.*

Partea 5: Construirea propriului modul (exercițiile 44–50)

Scop: transformați funcțiile într-un modul pe care alte programe îl pot importa, la fel ca requests. La final, directorul vostru ar trebui să arate astfel:

```
web_lab/
 ├── webtools.py    # modulul vostru: toate funcțiile reutilizabile
 ├── main.py        # programul care importă webtools
 ├── report.csv     # creat în exercițiul 49
 └── report.json    # creat în exercițiul 50
```

Exercițiul 44. Crearea unui modul. Mutați fetch, get_status, get_title și security_headers într-un fișier nou, webtools.py. În main.py, scrieți import webtools și apelați webtools.get_title(webtools.fetch(BASE_URL).text).

```
#webtools.py:
def fetch(url, timeout=TIMEOUT):
    """Returnează răspunsul pentru url, folosind antetele implicite."""
    return requests.get(url, timeout=timeout, headers=DEFAULT_HEADERS)
```

```
def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

def get_title(html):
    """Returnează titlul paginii HTML sau un șir gol dacă lipsește."""
    start, end = html.find("<title>"), html.find("</title>")
    if start == -1 or end == -1:
        return ""
    return html[start + len("<title>"):end].strip()

def security_headers(url):
    """Returnează {antet: True/False} pentru cele 5 antete de
securitate."""
    headers = fetch(url).headers
    return {name: name in headers for name in SECURITY_HEADERS}

def score_headers(results):
    """Returnează scorul antetelor, de forma '3/5'."""
    return f"{sum(results.values())}/{len(results)}"

def check_paths(base, paths):
    """Returnează {cale: cod_de_stare} pentru fiecare cale din
listă."""
    results = {}
    for path in paths:
        try:
            results[path] = get_status(base + path)
        except requests.RequestException:
            results[path] = None
        time.sleep(1)
    return results

def fetch_robots(base):
    """Returnează textul /robots.txt sau None dacă lipsește."""
```

```
    try:
        r = fetch(base + "/robots.txt")
    except requests.RequestException:
        return None
    return r.text if r.status_code == 200 else None

def disallowed_paths(robots_text):
    """Returnează lista valorilor Disallow din robots.txt."""
    if robots_text is None:
        return []
    paths = []
    for line in robots_text.splitlines():
        if line.lower().startswith("disallow:"):
            value = line.split(":", 1)[1].strip()
            if value:
                paths.append(value)
    return paths

def extract_links(html):
    """Returnează legăturile din html, fără duplicate."""
    return list(dict.fromkeys(re.findall(r'href="([^"]+)"', html)))

def split_links(links, domain):
    """Împarte legăturile în (interne, externe) față de domeniu."""
    internal, external = [], []
    for link in links:
        full = urljoin("https://" + domain, link)
        if urlparse(full).netloc == domain:
            internal.append(full)
        else:
            external.append(full)
    return internal, external

def resolve(hostname):
```

```
    """Returnează adresa IPv4 a domeniului."""
    return socket.gethostbyname(hostname)


def cert_days_left(hostname):
    """Returnează zilele rămase până la expirarea certificatului
TLS."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as
sock:
        with context.wrap_socket(sock, server_hostname=hostname) as
tls:
            cert = tls.getpeercert()
    return int((ssl.cert_time_to_seconds(cert["notAfter"]) -
time.time()) // 86400)
```

```
#main.py:
import webtools


print(webtools.get_title(webtools.fetch(webtools.BASE_URL).text))
```

Exercițiul 45. Importarea anumitor nume. În `main.py`, treceți la `from webtools import get_title, security_headers`. Într-un comentariu, explicați ce s-ar întâmpla dacă `main.py` ar avea și o funcție proprie numită `get_title`.

```
#main.py:
from webtools import get_title, security_headers, check_paths,
write_csv_report, site_report, fetch
```

Exercițiul 46. Blocul main. Adăugați acest bloc la finalul fișierului `webtools.py`:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rulați `python webtools.py`, apoi `python main.py`. Explicați de ce autotestul rulează doar în primul caz.

```
#webtools.py:
```

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Exercițiul 47. Constante de modul. Adăugați constanta `DEFAULT_HEADERS = {"User-Agent": "WebLab-<numele vostru>"}` în `webtools.py` și folosiți-o în `fetch()`. În `main.py`, importați constanta și afișați-o.

```
#webtools.py:
DEFAULT_HEADERS = {"User-Agent": "WebLab-Iurco Mihai"}

SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]
```

```
#main.py:
from webtools import DEFAULT_HEADERS

print(DEFAULT_HEADERS)
```

Exercițiul 48. Argumente din linia de comandă. Folosiți `argparse` în `main.py`, astfel încât programul să ruleze ca `python main.py https://cybercor.org`. Transmiteți argumentul URL funcțiilor voastre, în loc să folosiți `BASE_URL`.

```
#main.py:
parser = argparse.ArgumentParser(description="Raport despre un site web")
parser.add_argument("url", help="adresa site-ului, de exemplu https://cybercor.org")
args = parser.parse_args()

print(get_title(fetch(args.url).text))
```

```
print(security_headers(args.url))
```

Exercițiul 49. Un raport CSV. Folosiți modulul `csv` pentru a salva rezultatul funcției `check_paths()` în `report.csv`, cu coloanele `path`, `status` și `checked_at`. Pentru oră, folosiți `datetime.now().isoformat()`.

```
#webtools.py:
def write_csv_report(results, filename="report.csv"):
    """Salvează {cale: status} în CSV, cu coloanele path, status și
checked_at."""
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["path", "status", "checked_at"])
        for path, status in results.items():
            writer.writerow([path, status, datetime.now().isoformat()])
```

```
#main.py:
results = check_paths(args.url, ["/", "/robots.txt", "/sitemap.xml"])
write_csv_report(results, "report.csv")
```

Exercițiul 50. Proiect final: raportul site-ului. Scrieți `site_report(url)` în `webtools.py`. Funcția trebuie să folosească funcțiile voastre pentru a colecta toate informațiile de mai jos, să le afișeze și să le salveze în `report.json`:

1. Codul de stare și URL-ul final
2. Titlul paginii
3. Adresa IP
4. Lanțul de redirecționări pornind de la `http://`
5. Scorul antetelor de securitate, de exemplu `3/5`
6. Zilele rămase până la expirarea certificatului
7. Numărul de legături interne și externe
8. Căile interzise din `robots.txt`

```
#webtools.py:
def site_report(url):
    """Colectează, afișează și salvează în report.json raportul site-
ului."""
```

```
    parsed = urlparse(url)
    host = parsed.netloc

    response = fetch(url)
    time.sleep(1)

    redirect_response = requests.get(
        "http://" + host, timeout=TIMEOUT, headers=DEFAULT_HEADERS
    )
    hops = redirect_response.history + [redirect_response]
    chain = []
    for i, step in enumerate(redirect_response.history):
        chain.append(f"{step.status_code} -> {hops[i + 1].url}")
    time.sleep(1)

    scores = security_headers(url)
    internal, external = split_links(extract_links(response.text),
host)
    disallowed =
disallowed_paths(fetch_robots(f"{parsed.scheme}://{host}"))

    try:
        days = cert_days_left(host)
    except (OSError, ssl.SSLError):
        days = None

    report = {
        "url": url,
        "status_code": response.status_code,
        "final_url": response.url,
        "title": get_title(response.text),
        "ip": resolve(host),
        "redirects": chain,
        "security_score": score_headers(scores),
        "cert_days_left": days,
```

```
        "internal_links": len(internal),
        "external_links": len(external),
        "disallowed_paths": disallowed,
    }

    print(f"=== Raport site: {url} ===")
    print("Cod de stare:", report["status_code"])
    print("Titlu:", report["title"])
    print("Adresă IP:", report["ip"])
    print("Redirecționări:", ", ".join(chain) if chain else "niciuna")
    print("Scor securitate:", report["security_score"])
    print("Certificat:", f"{days} de zile rămase" if days is not None
else "indisponibil")
    print("Legături:", f"{len(internal)} interne, {len(external)}
externe")
    print("Căi interzise:", ", ".join(disallowed) if disallowed else
"niciuna")

    with open("report.json", "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2, ensure_ascii=False)
    print("Salvat în report.json")
    return report
```

```
#main.py:
site_report(args.url)
```

Un exemplu de rezultat (valorile voastre vor fi diferite):

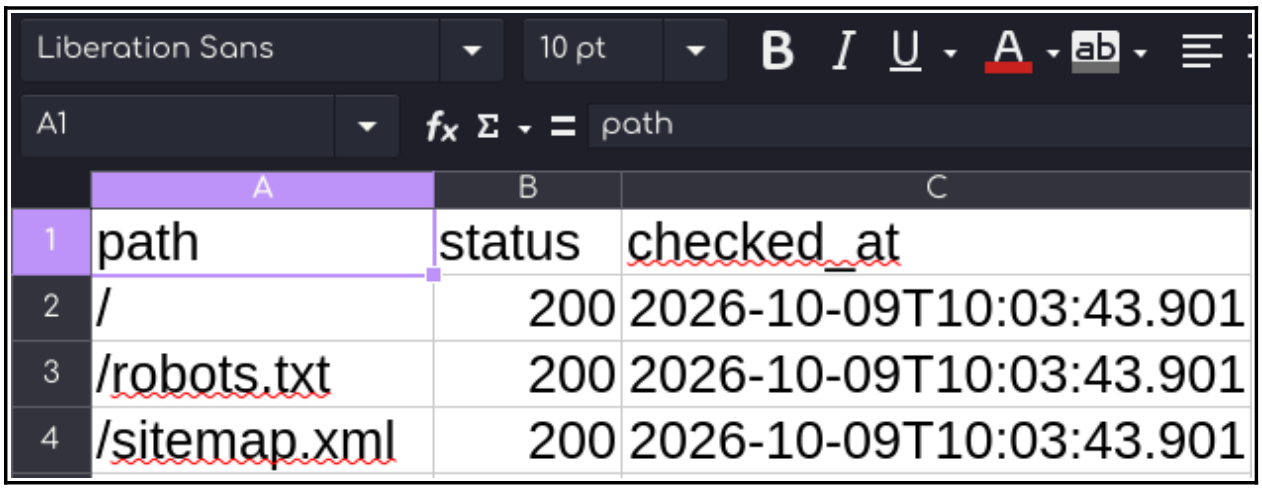
```
=== Raport site: https://cybercor.org ===
Cod de stare:     200
Titlu:          CYBERCOR
Adresă IP:       203.0.113.10
Redirecționări:   301 -> https://cybercor.org/
Scor securitate:  3/5
Certificat:       74 de zile rămase
Legături:         38 interne, 12 externe
Căi interzise:    /admin, /private
Salvat în report.json
```

> **Bonus:** instalați beautifulsoup4 și rescrieți get_title() și extract_links() folosind această bibliotecă. Comparați cât de mult mai scurte și mai sigure devin.

```
 mikepulowsky@fedora  ~/UTM/Programming Techniques/web_lab  python main.py https://cybercor.org
CYBERCOR
{'User-Agent': 'WebLab-Iurco Mihai'}
CYBERCOR
{'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content
-Type-Options': True, 'Referrer-Policy': True}
=== Raport site: https://cybercor.org ===
Cod de stare: 200
Titlu: CYBERCOR
Adresă IP: 104.21.91.71
Redirecționări: 301 -> https://cybercor.org/
Scor securitate: 3/5
Certificat: 55 de zile rămase
Legături: 1 interne, 0 externe
Căi interzise: niciuna
Salvat în report.json
 mikepulowsky@fedora  ~/UTM/Programming Techniques/web_lab  ls
headers.json  part1_imports.py  part3_functions.py  __pycache__  report.json
main.py       part2_methods.py  part4_modules.py    report.csv   webtools.py
 mikepulowsky@fedora  ~/UTM/Programming Techniques/web_lab
```

nvim report.json

```
1  {
1    "url": "https://cybercor.org",
2    "status_code": 200,
3    "final_url": "https://cybercor.org/",
4    "title": "CYBERCOR",
5    "ip": "104.21.91.71",
6    "redirects": [
7      "301 -> https://cybercor.org/"
8    ],
9    "security_score": "3/5",
10   "cert_days_left": 55,
11   "internal_links": 1,
12   "external_links": 0,
13   "disallowed_paths": []
14 }
```



	A	B	C
1	path	status	checked_at
2	/	200	2026-10-09T10:03:43.901
3	/robots.txt	200	2026-10-09T10:03:43.901
4	/sitemap.xml	200	2026-10-09T10:03:43.901

Predare și notare

Predați întregul director `web_lab` ca o singură arhivă `.zip`, numită `web_lab_<numele_vostru>.zip`.

Înainte de predare

- [ ] Fiecare fișier rulează fără erori de la început până la sfârșit.
- [ ] Fiecare cerere folosește un timeout, iar între cererile repetate există `time.sleep(1)`.
- [ ] Fiecare funcție, începând cu Partea 3, are un docstring.
- [ ] Toate cerințele de tip "explicați într-un comentariu" au răspuns în cod.
- [ ] `python main.py https://cybercor.org` afișează raportul complet și creează `report.json`.
- [ ] Directorul conține `report.csv` și `report.json`.

Notare

Partea	Exerciții	Puncte
Partea 1: Importuri	1–8	10
Partea 2: Metode și atribute	9–20	20
Partea 3: Propriile funcții	21–34	25
Partea 4: Module din biblioteca standard	35–43	20
Partea 5: Propriul modul și proiectul final	44–50	20
Respectarea regulilor de lucru	toate	5
Total		**100**

Exercițiul bonus (BeautifulSoup): până la 5 puncte în plus.